

# **CLOUD COMPUTING**

## **ASSIGNMENT#02**

Name: **Uzair Ahmad**

Enrollment ID: **913**

Semester: **6<sup>th</sup>**

Email: [ozyr.uc@gmail.com](mailto:ozyr.uc@gmail.com)

Submitted to: **M. Waqas Sir**

**Department Of Software Engineering  
University Of Malakand**

# Title:

## Designing a Cloud-Based Solution for a Growing Startup

### Scenario

A startup named “ShopEase” is launching an online e-commerce platform in Pakistan. Initially, it expects low traffic, but it plans to expand rapidly across multiple cities.

The startup faces the following challenges:

- Limited budget for IT infrastructure
- Need for high availability during sales/events
- Concerns about data security and user privacy
- Uncertain future growth (traffic may increase suddenly)
- Wants to avoid long-term vendor dependency

### Solution:

#### Cloud Architecture:

A Microservices Architecture deployed on Managed Kubernetes would be most effective as according to scenario and requirements.

- **Scalability:** We will use a managed Kubernetes service (like Google Kubernetes Engine or AWS EKS). This allows ShopEase to scale “Pods” (individual app components) up or down automatically based on CPU usage.
- **Security And Growth:** Use a Managed Relational Database (PostgreSQL or MySQL) with “High Availability” (HA) enabled. This ensures that if one database server fails, a standby kicks in immediately.
- **Storage:** Use Object Storage (like S3 or Google Cloud Storage) for product images and user upload. It is incredibly cheap and scales infinitely.

#### Core Challenges:

1. **Limited Budget: “Pay-As-You-Go Model”** Start with "Pre-emptible" or "Spot" instances for non-critical tasks to save up to 80% on costs. Use "Serverless" functions for background tasks (like sending emails) so you only pay when code actually runs.
2. **High Availability: “Multi-Zone Deployment”** Distribute the application across at least two geographical zones. If one data center goes down, the platform stays online. A Content Delivery Network (CDN) should be used to cache images locally in Pakistan to reduce latency.
3. **Data Security: “Encryption at Rest & Transit”** Use SSL/TLS for all user data. Implement IAM (Identity and Access Management) with the "Principle of Least Privilege" to ensure employees

only access what they absolutely need.

4. **Sudden Growth: “Horizontal Auto-Scaling”** Instead of buying a bigger server, the system automatically adds more small servers during a sale and removes them when the traffic drops.
5. **Vendor Dependency: “Containerization (Docker)”** By "wrapping" the application in Docker containers, ShopEase can move its entire platform from one provider to another (e.g., from AWS to Azure) with minimal code changes.

## **Implementation:**

1. **Phase 1 (The MVP):** Deploy a monolithic application on a single small cluster. Use a managed database to reduce the need for a dedicated IT admin.
2. **Phase 2 (Growth):** Break the monolith into microservices (e.g., separate services for "Cart," "Payments," and "User Profiles"). This allows you to scale only the parts of the app under heavy load.
3. **Phase 3 (Optimization):** Implement Infrastructure as Code (IaC) using tools like Terraform. This allows ShopEase to "code" their infrastructure, making it easy to replicate or migrate.